Cloud Computing Labs:-

Lab 2:-

Task 1: Install VM using virtual box or VMware workstation. Establish atleast two ubuntu-based VMs. Execute an MPI-hello-world application using two VMs.

Task 2: Create a ubuntu VM on Amazon AWS.

Creating ec2 instance of ubuntu

Do aws thing
Create a mykey pair
Command in power shell:- **ssh -i mykey.pem@Ubuntu<Publicipv4>**

Lab 3:-

Opened: Wednesday, 28 January 2026, 12:00 AM
Due: Monday, 2 February 2026, 11:38 PM

Task 1: Write a dynamoDB code using nodejs (create a database, delete a database, and modify a field)

Task 2: Write the same operations using golang

Create iam user in aws
Get accesskey secret key region
Open Vs Code
In terminal **-> aws–configure**
Enter details
Commands :- npm init -y
npm install @aws-sdk/client-dynamodb @aws-sdk/lib-dynamodb

[createTable.js](#):-

```
import { DynamoDBClient, CreateTableCommand } from
"@aws-sdk/client-dynamodb";
```

```
const client = new DynamoDBClient({ region: "us-east-1" });

async function createTable() {
  await client.send(new CreateTableCommand({
    TableName: "Students_bcs161",
    AttributeDefinitions: [
      { AttributeName: "id", AttributeType: "S" }
    ],
    KeySchema: [
      { AttributeName: "id", KeyType: "HASH" }
    ],
    BillingMode: "PAY_PER_REQUEST"
  }));

  console.log("✅ Table Created");
}

createTable();
```

[insertdata.go]

```
package main

import (
	"context"
	"fmt"

	"github.com/aws/aws-sdk-go-v2/config"
	"github.com/aws/aws-sdk-go-v2/service/dynamodb"
	"github.com/aws/aws-sdk-go-v2/service/dynamodb/types"
)

func main() {
	cfg, _ := config.LoadDefaultConfig(context.TODO())
	client := dynamodb.NewFromConfig(cfg)
```

```go
	items := []map[string]types.AttributeValue{
		{
			"id":   &types.AttributeValueMemberS{Value: "1"},
			"cgpa": &types.AttributeValueMemberN{Value: "8.2"},
		},
		{
			"id":   &types.AttributeValueMemberS{Value: "2"},
			"cgpa": &types.AttributeValueMemberN{Value: "8.5"},
		},
	}

	for _, item := range items {
		_, err := client.PutItem(context.TODO(),
&dynamodb.PutItemInput{
			TableName: awsString("Students_bcs161"),
			Item:      item,
		})

		if err != nil {
			fmt.Println("Error:", err)
			return
		}
	}

	fmt.Println("✅ Data Inserted")
}

func awsString(s string) *string {
	return &s
}
```

Lab4 ➖

Opened: Wednesday, 4 February 2026, 12:00 AM
Due: Saturday, 7 February 2026, 11:59 PM

Task 1: Explore docker containers (familiarize popular docker commands)

Task 2: Create a docker container on your name using DockerFile

Task 3: Expose a docker container after hosting a nodejs application.

Docker Commands:-

sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
docker images
docker pull ubuntu
docker ps
docker ps -a
docker run -it ubuntu bash
docker stop <container_id>
docker rm <container_id>
docker rmi <image_id>

2)
mkdir node_docker_app_vivek
cd node_docker_app_vivek

app.js

```
const http = require('http');


const server = http.createServer((req, res) => {
```

```
  res.end("Hello from Vandrangi Sai Vivek 2023BCS0161");
});

server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Dockerfile

```
FROM node:18

WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

docker build -t node_app .
docker run -p 3000:3000 node_app

LAB 5:-

Opened: Wednesday, 11 February 2026, 12:00 AM
Due: Saturday, 14 February 2026, 11:59 PM

Task 1: Install minikube; start it and explore the dashboard.

Task 2: Deploy an application written for a container using minikube (using docker driver); and verify the status of pods using commands. (Explore deployment-related YAML file -- apiVersion, kind, metadata, and spec (replica, selector, template (spec(container details))); Explore deployment commands)

Task 3: Write an ML-as-a-service on a minikube-based container. Expose the service and share the prediction output.

Install Minikube

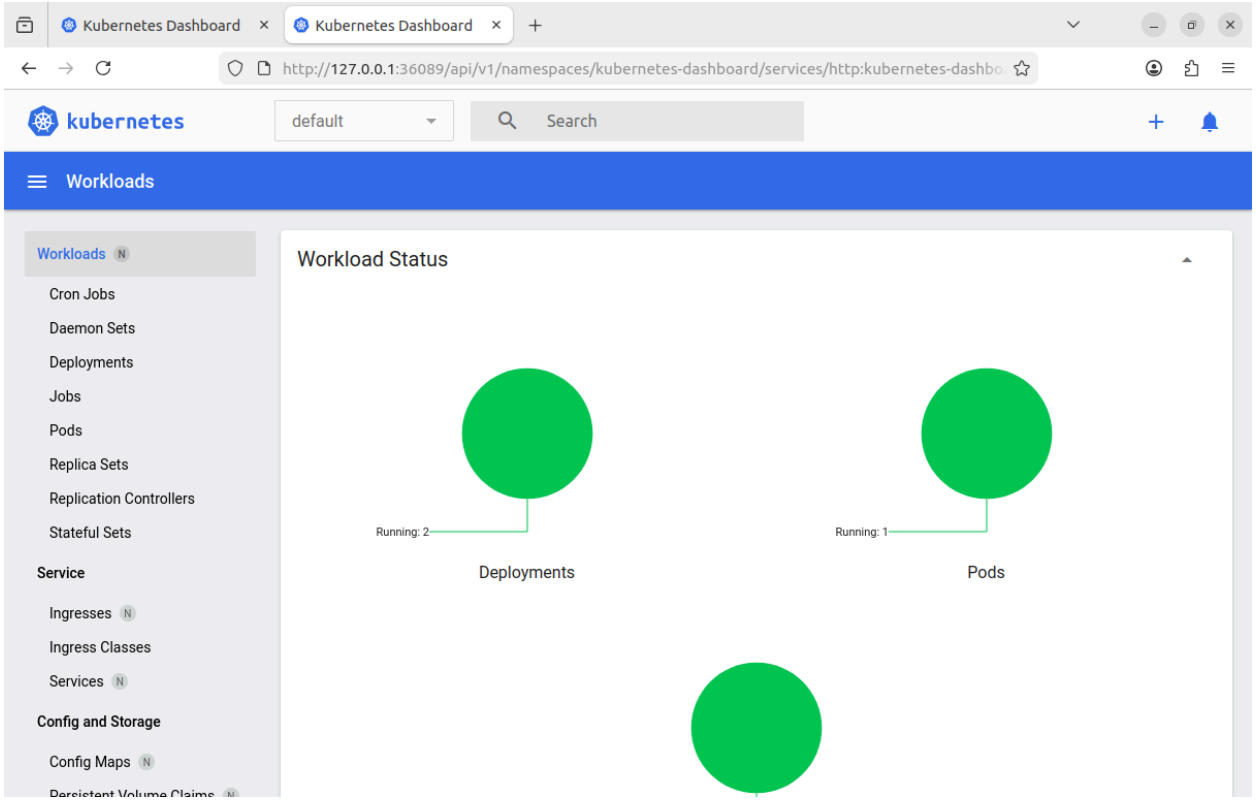
curl -LO
https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

minikube version

minikube start –driver=docker

minikube status
kubectl get nodes

minikube dashboard



2)

minikube start --driver=docker

nano app.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end("Hello from Kubernetes Deployment");
});

server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

nano Dockerfile

```
FROM node:18

WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

eval $(minikube docker-env)

docker build -t node_app .

nano deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-app-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
```

```
      app: node-app
  template:
    metadata:
      labels:
        app: node-app
    spec:
      containers:
      - name: node-container
        image: node_app
        imagePullPolicy: Never
        ports:
        - containerPort: 3000
```

kubectl apply -f deployment.yaml
kubectl get pods
kubectl describe pod <pod_name>
kubectl logs <pod_name>
kubectl get pods -w
kubectl get deployments
kubectl scale deployment node-app-deployment --replicas=3
kubectl delete deployment node-app-deployment

3)
app.js

```
const express = require('express');

const app = express();
app.use(express.json());

// Dummy ML model (replace later if needed)
function predict(x) {
    return x * 2;
```

```
}

app.get('/', (req, res) => {
    res.send("ML Service Running (Node.js)");
});

app.post('/predict', (req, res) => {
    const x = req.body.input;
    const result = predict(x);
    res.json({ prediction: result });
});

app.listen(5000, '0.0.0.0', () => {
    console.log("Server running on port 5000");
});

npm init -y
npm install express

FROM node:18

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
```

```
EXPOSE 5000

CMD ["node", "app.js"]
```

```
eval $(minikube docker-env)
docker build -t ml-node-service .
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ml-node-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ml-node-app
  template:
    metadata:
      labels:
        app: ml-node-app
    spec:
      containers:
      - name: ml-node-container
        image: ml-node-service
        imagePullPolicy: Never
        ports:
        - containerPort: 5000
```

```
kubectl apply -f ml-node-deployment.yaml
kubectl get pods
kubectl expose deployment ml-node-deployment
--type=NodePort --port=5000
minikube service ml-node-deployment --url
curl -X POST http://<URL>/predict \
-H "Content-Type: application/json" \
-d '{"input": 7}'
```

Lab6:-

Opened: Wednesday, 25 February 2026, 12:00 AM
Due: Monday, 2 March 2026, 11:59 PM

VMWare vsphere lab

Task 1 - Create a linear regression prediction service using node.js or golang. Deploy the service as docker container on top of minikube orchestration platform. Utilize the kubernetes-related commands to deploy and describe services.

Task 2 - Explore VMWare vsphere product (data automation lab) -- refer task 1 to task 4 of our class notes

Task 3 - Clone a VM on you name using scripts -- refer task 9 of our class notes.

1)

```
mkdir mlaas
cd mlaas
const express = require('express');
const app = express();

app.use(express.json());

app.get('/', (req, res) => {
```

```
    res.send("ML working");
});

app.post('/predict', (req, res) => {
    const x = req.body.input;
    const y = 2 * x + 3;
    res.json({ prediction: y });
});

app.listen(3000, '0.0.0.0', () => {
    console.log("server running on port 3000");
});

npm init -y
npm install express

node app.js

curl -X POST http://localhost:3000/predict \
-H "Content-Type: application/json" \
-d '{"input":5}'

FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

```
minikube start --driver=docker

eval $(minikube docker-env)


docker build -t ml-node-service .

apiVersion: apps/v1
kind: Deployment
metadata:
  name: ml-deployment

spec:
  replicas: 2
  selector:
    matchLabels:
      app: ml-app

  template:
    metadata:
      labels:
        app: ml-app

    spec:
      containers:
      - name: ml-container
        image: ml-node-service
        imagePullPolicy: Never
        ports:
```

```
        - containerPort: 3000
```

```
kubectl apply -f ml-deployment.yaml
kubectl get pods
```

```
apiVersion: v1
kind: Service
metadata:
  name: ml-service

spec:
  type: NodePort
  selector:
    app: ml-app

  ports:
  - port: 3000
    targetPort: 3000
```

```
kubectl apply -f ml-service.yaml
kubectl get services
kubectl get services
{"prediction":23}
```

2)VMWare Vsphere

pwsh

```
$env:http_proxy=""
$env:https_proxy=""
Set-PowerCLIConfiguration -InvalidCertificateAction Ignore
-Confirm:$false

Connect-VIServer vc-mgmt-a.site-a.vcf.lab -User
administrator@vsphere.local

View Password in password.txt
Get-VM
Get-VMHost
Get-Datacenter
Get-Datacenter
Get-Folder
Get-Cluster
Get-VIServer
Get-VM | Select Name, PowerState
```

3)

```
$env:http_proxy=""
$env:https_proxy=""
Set-PowerCLIConfiguration -InvalidCertificateAction Ignore
-Confirm:$false
Connect-VIServer vc-mgmt-a.site-a.vcf.lab -User
administrator@vsphere.local
Get-VM
Get-VM "acct-web01" | New-VM -Name "Vivek_Clone"
-VMHost (Get-VMHost | Select -First 1)
```

Get-VM Vivek_Clone
Start-VM Vivek_Clone
Get-VM

Lab 7)

Opened: Wednesday, 11 March 2026, 12:00 AM
Due: Tuesday, 17 March 2026, 11:55 PM

Task 1: Write a microservice using a docker container (on a minikube platform) that shares the GPS location of your class room.

Task 2: Monitor the performance of containers using a prometheus tool.

Task 3: Enable a CI/CD pipeline workflow using GitHub actions.

1. Python Microservice (app.py)

```
from flask import Flask, jsonify
import requests

app = Flask(__name__)

@app.route('/')
def get_location():
    response = requests.get("http://ip-api.com/json/")
    data = response.json()

    result = {
        "city": data.get("city"),
        "region": data.get("regionName"),
```

```
        "latitude": data.get("lat"),
        "longitude": data.get("lon")
    }
    return jsonify(result)

app.run(host="0.0.0.0", port=5000)
```

2. Requirements.txt

```
flask
requests
```

3. Dockerfile

```
FROM python:3.9

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

```
docker build -t gps-service .
docker run -p 5000:5000 gps-service

minikube start –driver=docker
```

```
kubectl create deployment gps-deployment
--image=gps-service
kubectl expose deployment gps-deployment
--type=NodePort --port=5000
minikube service gps-deployment
```

If error

```
kubectl delete deployment gps-deployment
kubectl run gps-deployment \
  --image=gps-service \
  --image-pull-policy=Never


minikube service gps-deployment
```

Prometheus start

```
docker run -d \
  --name=cadvisor \
  -p 8080:8080 \
  --volume=/:/rootfs:ro \
  --volume=/var/run:/var/run:ro \
  --volume=/sys:/sys:ro \
  --volume=/var/lib/docker/:/var/lib/docker:ro \
  gcr.io/cadvisor/cadvisor:latest
```

```
nano prometheus.yml

global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'cadvisor'
    static_configs:
      - targets: ['localhost:8080']
```

```
docker run -d -p 9090:9090 \
  -v
$(pwd)/prometheus.yml:/etc/prometheus/prometheus.yml \
  prom/prometheus
```

git init
git add .
git commit -m "initial commit"
git branch -M main
git remote add origin <repo-url>
git push -u origin main

```
mkdir -p .github/workflows
```

```
nano .github/workflows/main.yml
name: CI/CD Pipeline

on:
  push:
    branches:+
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Build Docker Image
        run: docker build -t gps-service .

git add .
git commit -m "added CI/CD pipeline"
git push origin main
```

Lab8 Terraform

Opened: Tuesday, 17 March 2026, 12:00 AM
Due: Monday, 23 March 2026, 11:59 PM

Task 1: Create a kubernetes resource using Terraform code.

Task 2: Install mongodb database on your local machine and explore commands to create, delete, and insert records of your B.Tech success stories.

Task 3: Create a CI/CD pipeline based on github actions that trigger Terraform code to start a service involving mongodb database.

```
minikube start --driver=docker
mkdir terraform-k8s
cd terraform-k8s
touch main.tf
```

```
provider "kubernetes" {
  config_path = "~/.kube/config"
}

resource "kubernetes_deployment" "nginx_deployment" {
  metadata {
    name = "nginx-deployment"
    labels = {
      app = "nginx"
    }
  }

  spec {
    replicas = 2

    selector {
      match_labels = {
        app = "nginx"
      }
    }

    template {
      metadata {
        labels = {
          app = "nginx"
        }
      }
```

```
      spec {
        container {
          image = "nginx:latest"
          name  = "nginx-container"

          port {
            container_port = 80
          }
        }
      }
    }
  }
}

resource "kubernetes_service" "nginx_service" {
  metadata {
    name = "nginx-service"
  }

  spec {
    selector = {
      app = "nginx"
    }

    port {
      port        = 80
      target_port = 80
    }

    type = "NodePort"
  }
}
```

```
terraform init
terraform apply

kubectl get pods
kubectl get svc
```

2)

```
sudo apt update
sudo apt install -y mongodb
sudo systemctl start mongodb
Mongo
use btech_stories
db.success.insertMany([
  { name: "Vivek", year: 2026, achievement: "Built ML project" },
  { name: "Arun", year: 2025, achievement: "Placed in MNC" }
])

db.success.find()
db.success.deleteOne({ name: "Arun" })
db.success.updateOne(
  { name: "Vivek" },
  { $set: { achievement: "Built ML + DevOps project" } }
)
```

3)

Create GitHub Repo

Create Workflow File .github/workflows/deploy.yml

```
name: Terraform Deploy

on:
  push:
    branches:
      - main

jobs:
  terraform:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout Code
      uses: actions/checkout@v3

    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v2
```

```
  - name: Terraform Init
    run: terraform init

  - name: Terraform Validate
    run: terraform validate

  - name: Terraform Apply
    run: terraform apply -auto-approve
```

```
git add .
git commit -m "CI/CD Terraform setup"
git push origin main
```

LAB 9 openfaas

Opened: Wednesday, 25 March 2026, 12:00 AM
Due: Tuesday, 31 March 2026, 11:59 PM

Task 1: Install OpenFaaS; (Next, create, deploy, and monitor (using Prometheus) serverless functions on a minikube cluster.

Task 2: Write a CI/CD pipeline using github actions that utilize Terraform to deploy and destroy OpenFaaS functions.

```
minikube start --driver=docker
curl -sSL https://cli.openfaas.com | sudo sh
faas-cli version
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm repo add openfaas https://openfaas.github.io/faas-netes/
helm repo update
kubectl create namespace openfaas
kubectl create namespace openfaas-fn
helm upgrade openfaas openfaas/openfaas \
  --install \
  --namespace openfaas \
  --set functionNamespace=openfaas-fn \
  --set generateBasicAuth=true
```

```
kubectl get pods -n openfaas
kubectl port-forward svc/gateway -n openfaas 8080:8080

PASSWORD=$(kubectl get secret -n openfaas basic-auth \
 -o jsonpath="{.data.basic-auth-password}" | base64 --decode)

echo $PASSWORD

faas-cli template pull https://github.com/openfaas/templates.git
faas-cli new hello-fn --lang node
module.exports = async (event, context) => {
  return "Hello from OpenFaaS 🚀";
}
faas-cli build -f hello-fn.yml
faas-cli push -f hello-fn.yml
faas-cli deploy -f hello-fn.yml
faas-cli invoke hello-fn
kubectl port-forward svc/prometheus -n openfaas 9090:9090
```

2)

main.tf

```
provider "kubernetes" {
  config_path = "~/.kube/config"
}

resource "kubernetes_deployment" "openfaas_fn" {
  metadata {
    name = "hello-fn"
    namespace = "openfaas-fn"
  }

  spec {
    replicas = 1

    selector {
      match_labels = {
        app = "hello-fn"
      }
    }

    template {
```

```
      metadata {
        labels = {
          app = "hello-fn"
        }
      }

      spec {
        container {
          name  = "hello-fn"
          image = "your-dockerhub-username/hello-fn:latest"

          port {
            container_port = 8080
          }
        }
      }
    }
  }
}
```

.github/workflows/openfaas.yml

Pipeline:-

```
name: OpenFaaS CI/CD

on:
  push:
    branches:
      - main

jobs:
  terraform:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout
      uses: actions/checkout@v3

    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v2
```

```
    - name: Terraform Init
      run: terraform init

    - name: Terraform Apply
      run: terraform apply -auto-approve

    - name: Terraform Destroy (on manual trigger)
      if: github.event_name == 'workflow_dispatch'
      run: terraform destroy -auto-approve
```

```
git add .
git commit -m "OpenFaaS CI/CD"
git push origin main
```